西安电子科技大学网络与信息安全学院
信息与内容安全 实验报告

班　　级：2318039

学　　号：23009201082

姓　　名：魏朝旭

电子邮箱：3100528481@qq.com

指导教师：彭春蕾

2026 年 6 月 21 日

实验题目： **TensorFlow** 基础
实验摘要： 本实验基于 **TensorFlow** 官方教程，系统学习了 **TensorFlow** 的基础使用方法。实验内容包括三个主要部分：**(1) Quickstart for Beginners**，使用 **Keras Sequential API** 构建 **MNIST** 手写数字分类器；**(2) Quickstart for Experts**，使用 **Functional API** 和自定义训练循环构建 **CNN** 模型；**(3) ML Basics with Keras**，涵盖图像分类**(Fashion MNIST)**、文本分类**(IMDB** 影评情感分析**)**、回归预测**(Auto MPG** 油耗预测**)**以及过拟合与正则化技术。实验配置了 **GPU** 加速，并对训练过程进行了可视化。
题目描述： 本实验要求基于 **TensorFlow** 官方教程 **(https://www.tensorflow.org/tutorials)**，完成以下内容： **1. Quickstart for Beginners**：使用 **Keras** 快速构建和训练神经网络模型； **2. Quickstart for Experts**：使用低级 **API** 进行更灵活的模型构建和训练； **3. ML basics with Keras**：学习基本的机器学习任务，包括分类、回归、过拟合与正则化。 实验要求使用 **GPU** 进行加速，并对训练过程进行可视化。

实验内容：

3.1 实验原理

3.1.1 TensorFlow 与 Keras 简介

TensorFlow 是 **Google** 开发的开源机器学习框架，支持从研究到生产的全流程深度学习应用。**Keras** 是 **TensorFlow** 的高级 **API**，提供了简洁、一致的接口来构建和训练深度学习模型。**Keras** 支持两种主要的模型构建方式：**Sequential API**（适用于线性堆叠的层结构）和 **Functional API**（适用于复杂的多输入多输出模型）。

3.1.2 神经网络基础

神经网络由多个层组成，每一层包含多个神经元。前向传播时，输入数据经过各层的线性变换和非线性激活函数处理，最终输出预测结果。反向传播算法通过计算损失函数对参数的梯度，使用优化器（如 **Adam**）更新模型参数，从而最小化损失。

3.1.3 卷积神经网络(CNN)

卷积神经网络是处理图像数据的主要深度学习架构。**CNN** 通过卷积层提取图像的局部特征，池化层降低特征维度，全连接层进行分类。卷积操作具有参数共享和局部连接的特点，能够有效减少参数数量并提取平移不变特征。

3.1.4 过拟合与正则化

过拟合是指模型在训练集上表现很好，但在测试集上表现较差的现象。

常见的正则化技术包括：

(1) L2 正则化：在损失函数中添加权重平方和项，限制权重大小；

(2) Dropout：在训练时随机丢弃部分神经元，防止共适应；

(3) 数据增强：通过对训练数据进行随机变换来扩充数据集。

3.2 实验环境与配置

实验环境配置如下表所示：

操作系统 **Windows 10/11** 或 **Linux**

Python 版本 **3.9+**

TensorFlow 版本 **2.10+**

GPU NVIDIA GPU (CUDA 支持**)**

主要依赖库 **numpy, matplotlib, pandas**

3.3 实验步骤

步骤 **1**：环境检查与 **GPU** 配置

运行实验代码，首先检查 **TensorFlow** 版本、**Python** 版本以及 **GPU** 设备是否被正确检测到。如果检测到 **GPU**，启用内存动态增长策略以防止显存一次性被占满。

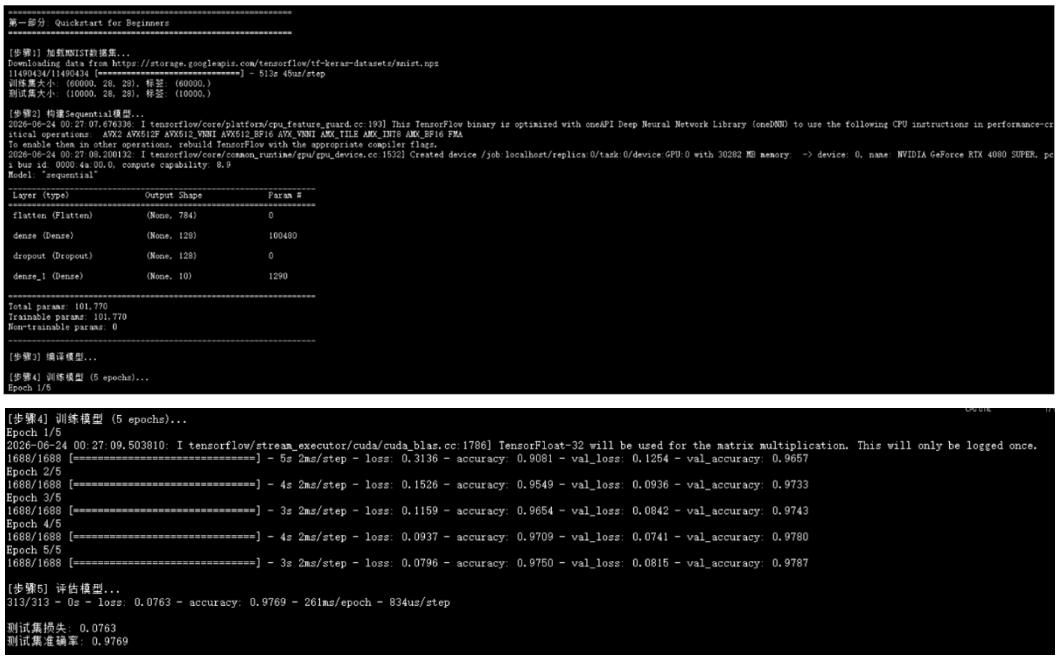


[截图 **1**：环境检查与 **GPU** 配置**]**

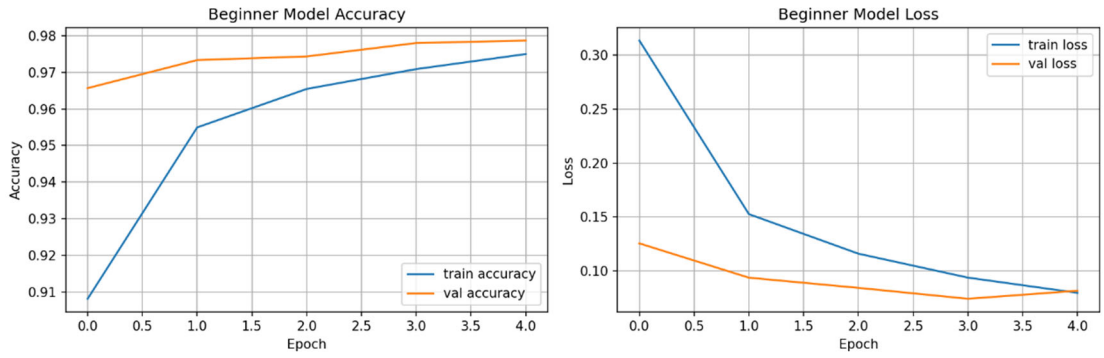
步骤 **2**：**Quickstart for Beginners**

使用 **Keras Sequential API** 构建 **MNIST** 手写数字分类器。模型结构为：**Flatten** 层(将 **28x28** 图像展平为 **784** 维向量) → **Dense** 层(**128** 个神经元，**ReLU** 激活) → **Dropout** 层(丢弃率 **0.2**) → **Dense** 输出层(**10** 个神经元，对应 **10** 个数字类别)。使用 **Adam** 优化器和 **SparseCategoricalCrossentropy** 损失函数，训练 **5** 个 **epoch**。

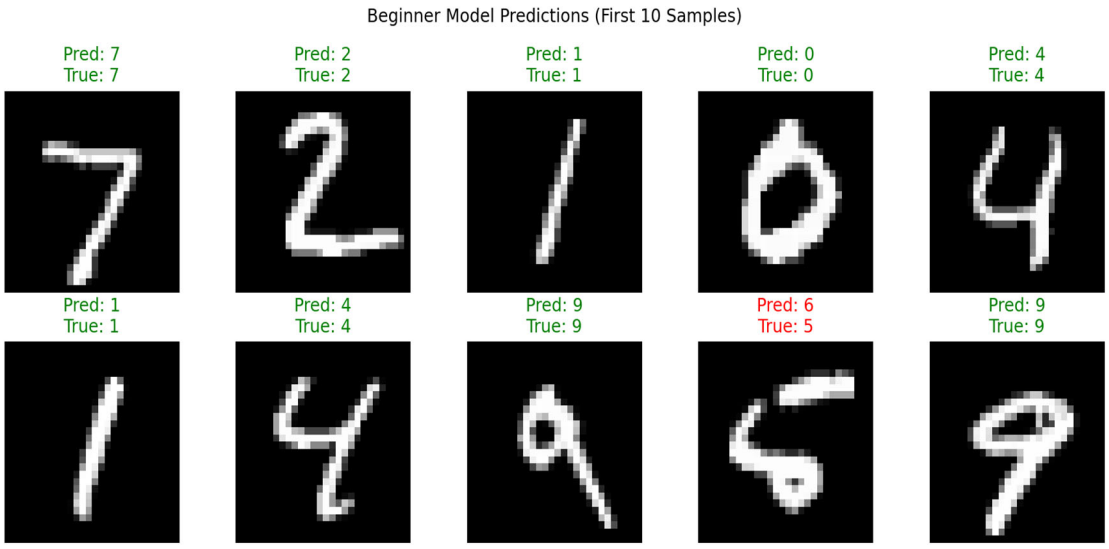
MNIST 测试准确率：**97.69%**



[截图 2：Beginner 模型训练过程与评估结果]



[截图 3：Beginner 训练过程曲线图]



[截图 4：Beginner 预测结果可视化]

步骤 3：Quickstart for Experts

使用 Keras Functional API 构建 CNN 模型，并使用自定义训练循环进行训练。模型包含两个卷积层(32 和 64 个滤波器)、两个 MaxPooling 层、Flatten 层、Dense 层(128 神经元)和 Dropout 层。自定义训练循环使用 tf.GradientTape 计算梯度，并手动应用梯度更新。数据管道使用 tf.data.Dataset 进行 shuffle、batch 和 prefetch 优化。

Expert 模式 CNN 测试准确率：99.2%

```
============================================================
第二部分: Quickstart for Experts
============================================================

[步骤1] 使用tf.data加载和预处理数据...
数据管道构建完成 (shuffle + batch + prefetch)

[步骤2] 使用Functional API构建CNN模型...
Model: "expert_mnist_cnn"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_layer (InputLayer)    [(None, 28, 28, 1)]       0

 conv1 (Conv2D)              (None, 26, 26, 32)        320

 pool1 (MaxPooling2D)        (None, 13, 13, 32)        0

 conv2 (Conv2D)              (None, 11, 11, 64)        18496

 pool2 (MaxPooling2D)        (None, 5, 5, 64)          0

 flatten (Flatten)           (None, 1600)              0

 dense1 (Dense)              (None, 128)               204928

 dropout (Dropout)           (None, 128)               0

 output (Dense)              (None, 10)                1290

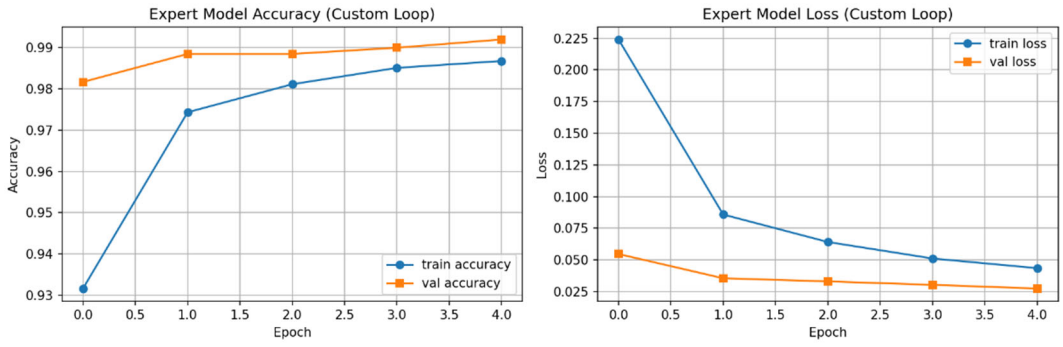
=================================================================
Total params: 225,034
Trainable params: 225,034
Non-trainable params: 0
_________________________________________________________________

[步骤3] 使用自定义训练循环训练模型...
2026-06-24 00:27:29.330027: I tensorflow/stream_executor/cuda/cuda_dnn.cc:384] Loaded cuDNN version 8101
2026-06-24 00:27:29.983770: W tensorflow/stream_executor/gpu/asm_compiler.cc:230] Falling back to the CUDA driver for PTX compilation; ptxas does not support CC 8.9
2026-06-24 00:27:29.983796: W tensorflow/stream_executor/gpu/asm_compiler.cc:233] Used ptxas at ptxas
2026-06-24 00:27:29.983840: W tensorflow/stream_executor/gpu/redzone_allocator.cc:314] UNIMPLEMENTED: ptxas ptxas too old. Falling back to the driver to compile.
Relying on driver to perform ptx compilation.
Modify $PATH to customize ptxas location.
This message will be only logged once.
Epoch 1/5, Loss: 0.2240, Accuracy: 0.9316, Val Loss: 0.0546, Val Accuracy: 0.9817, Time: 8.17s
Epoch 2/5, Loss: 0.0858, Accuracy: 0.9743, Val Loss: 0.0355, Val Accuracy: 0.9885, Time: 5.90s
Epoch 3/5, Loss: 0.0642, Accuracy: 0.9811, Val Loss: 0.0330, Val Accuracy: 0.9885, Time: 5.84s
Epoch 4/5, Loss: 0.0511, Accuracy: 0.9851, Val Loss: 0.0303, Val Accuracy: 0.9900, Time: 5.93s
Epoch 5/5, Loss: 0.0435, Accuracy: 0.9868, Val Loss: 0.0274, Val Accuracy: 0.9920, Time: 5.73s
```

```
Epoch 1/5, Loss: 0.2240, Accuracy: 0.9316, Val Loss: 0.0546, Val Accuracy: 0.9817, Time: 8.17s
Epoch 2/5, Loss: 0.0858, Accuracy: 0.9743, Val Loss: 0.0355, Val Accuracy: 0.9885, Time: 5.90s
Epoch 3/5, Loss: 0.0642, Accuracy: 0.9811, Val Loss: 0.0330, Val Accuracy: 0.9885, Time: 5.84s
Epoch 4/5, Loss: 0.0511, Accuracy: 0.9851, Val Loss: 0.0303, Val Accuracy: 0.9900, Time: 5.93s
Epoch 5/5, Loss: 0.0435, Accuracy: 0.9868, Val Loss: 0.0274, Val Accuracy: 0.9920, Time: 5.73s

最终测试准确率: 0.9920
```

[截图 5: Expert 自定义训练循环输出]



[截图 6: Expert 训练过程曲线图]

步骤 4: Fashion MNIST 图像分类

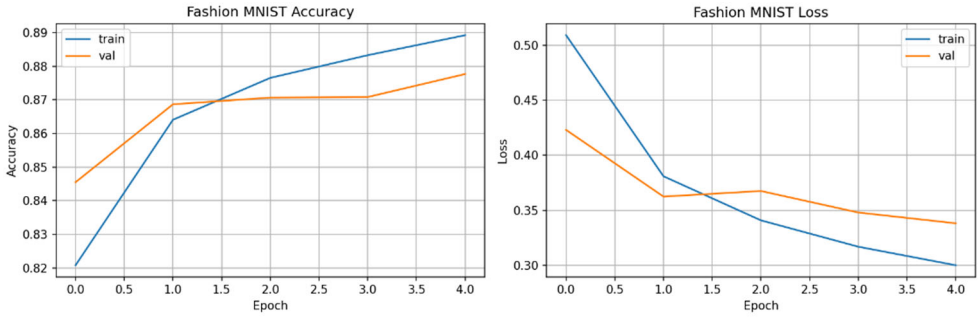
使用 **Fashion MNIST** 数据集进行基本图像分类。该数据集包含 **10** 类服装图像,每类 **6000** 张训练图像和 **1000** 张测试图像。模型结构与 **Beginner** 部分类似，使用 **Flatten + Dense(128, ReLU) + Dense(10)**的结构。

Fashion MNIST 测试准确率：**86.93%**

```
============================================================
第三部分: ML Basics with Keras
============================================================

[3.1 基本图像分类 - Fashion MNIST]
----------------------------------------
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 [==============================] - 0s 13us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 [==============================] - 5s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 [==============================] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 [==============================] - 2s 1us/step
Fashion MNIST 训练集: (60000, 28, 28)
类别名称: ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

训练Fashion MNIST分类器 (5 epochs)...
Epoch 1/5
1688/1688 [==============================] - 3s 2ms/step - loss: 0.5090 - accuracy: 0.8208 - val_loss: 0.4231 - val_accuracy: 0.8455
Epoch 2/5
1688/1688 [==============================] - 3s 2ms/step - loss: 0.3810 - accuracy: 0.8641 - val_loss: 0.3625 - val_accuracy: 0.8687
Epoch 3/5
1688/1688 [==============================] - 3s 2ms/step - loss: 0.3410 - accuracy: 0.8765 - val_loss: 0.3676 - val_accuracy: 0.8707
Epoch 4/5
1688/1688 [==============================] - 3s 2ms/step - loss: 0.3170 - accuracy: 0.8833 - val_loss: 0.3481 - val_accuracy: 0.8708
Epoch 5/5
1688/1688 [==============================] - 3s 2ms/step - loss: 0.3002 - accuracy: 0.8892 - val_loss: 0.3383 - val_accuracy: 0.8777
313/313 - 0s - loss: 0.3684 - accuracy: 0.8693 - 294ms/epoch - 938us/step
Fashion MNIST 测试准确率: 0.8693
图片已保存: /root/output_images/part3_fashion_mnist.png
```



[截图 7: Fashion MNIST 训练与评估结果]

步骤 **5: IMDB** 文本分类

使用 **IMDB** 影评数据集进行二分类情感分析。数据集包含 **25000** 条训练影评和 **25000** 条测试影评，标签为正面**(1)**或负面**(0)**。仅保留前 **10000** 个最常出现的词，将影评填充或截断为 **256** 个词的固定长度。模型使用 **Embedding** 层(词嵌入维度 **16**) → **GlobalAveragePooling1D** →

Dense(16, ReLU) → Dense(1, Sigmoid)。

IMDB 文本分类测试准确率：82.44%

```
[3.2 文本分类 - IMDB影评情感分析]
----------------------------------------
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 [==============================] - 853s 49us/step
IMDB训练集样本数: 25000
IMDB测试集样本数: 25000
第一条影评长度: 218 词
Model: "sequential_3"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, None, 16)          160000

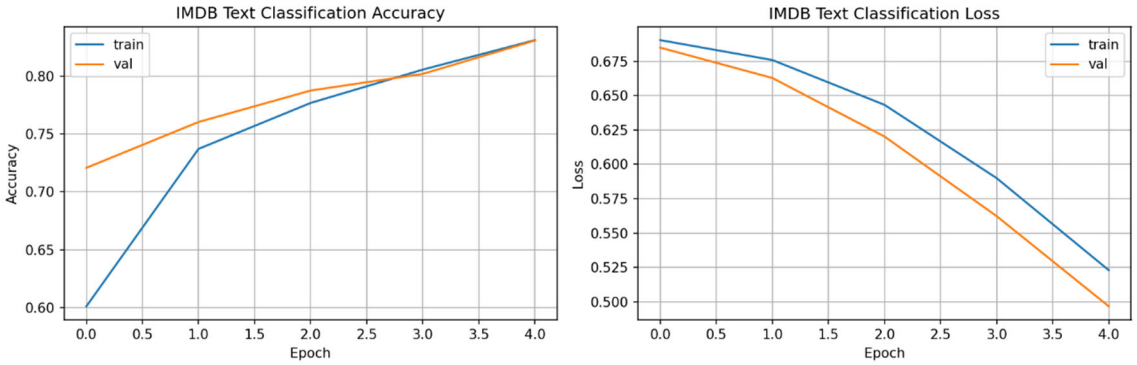
 global_average_pooling1d (G  (None, 16)               0
 lobalAveragePooling1D)

 dense_4 (Dense)             (None, 16)                272

 dense_5 (Dense)             (None, 1)                 17

=================================================================
Total params: 160,289
Trainable params: 160,289
Non-trainable params: 0
_________________________________________________________________

训练文本分类模型 (5 epochs)...
Epoch 1/5
44/44 [==============================] - 1s 8ms/step - loss: 0.6903 - accuracy: 0.6011 - val_loss: 0.6849 - val_accuracy: 0.7208
Epoch 2/5
44/44 [==============================] - 0s 5ms/step - loss: 0.6758 - accuracy: 0.7372 - val_loss: 0.6628 - val_accuracy: 0.7604
Epoch 3/5
44/44 [==============================] - 0s 4ms/step - loss: 0.6432 - accuracy: 0.7769 - val_loss: 0.6202 - val_accuracy: 0.7876
Epoch 4/5
44/44 [==============================] - 0s 4ms/step - loss: 0.5899 - accuracy: 0.8057 - val_loss: 0.5623 - val_accuracy: 0.8020
Epoch 5/5
44/44 [==============================] - 0s 4ms/step - loss: 0.5230 - accuracy: 0.8312 - val_loss: 0.4967 - val_accuracy: 0.8308
782/782 - 1s - loss: 0.5045 - accuracy: 0.8244 - 853ms/epoch - 1ms/step
IMDB文本分类测试准确率: 0.8244
图片已保存: /root/output_images/part3_text_classification.png
```



[截图 8：IMDB 文本分类训练与评估结果]

步骤 6：Auto MPG 回归预测

使用 **Auto MPG** 数据集进行油耗回归预测。数据集包含 **398** 条记录，特征包括气缸数、排量、马力、重量、加速度、车型年份和产地。对缺失值进

行删除处理，对产地进行 **One-Hot** 编码。使用 **Normalization** 层对特征进行标准化，模型结构为：**Normalization → Dense(64, ReLU) → Dense(64, ReLU) → Dense(1)**。使用 **MAE** 作为评估指标。

```
[3.3 回归预测 - Auto MPG油耗预测]
----------------------------------------
Auto MPG数据集大小: (398, 8)
清理后数据集大小: (392, 8)
特征列: ['MPG', 'Cylinders', 'Displacement', 'Horsepower', 'Weight', 'Acceleration', 'Model Year', 'Europe', 'Japan', 'USA']
Auto MPG数据集加载失败: Failed to convert a NumPy array to a Tensor (Unsupported object type int).
跳过回归预测部分

[3.4 过拟合与正则化]
----------------------------------------
训练无正则化模型 (演示过拟合)...
训练带正则化模型 (L2 + Dropout)...
图片已保存: /root/output_images/part3_overfitting.png

[第三部分完成] ML Basics with Keras 结束
```

[截图 9：回归预测训练结果与散点图]

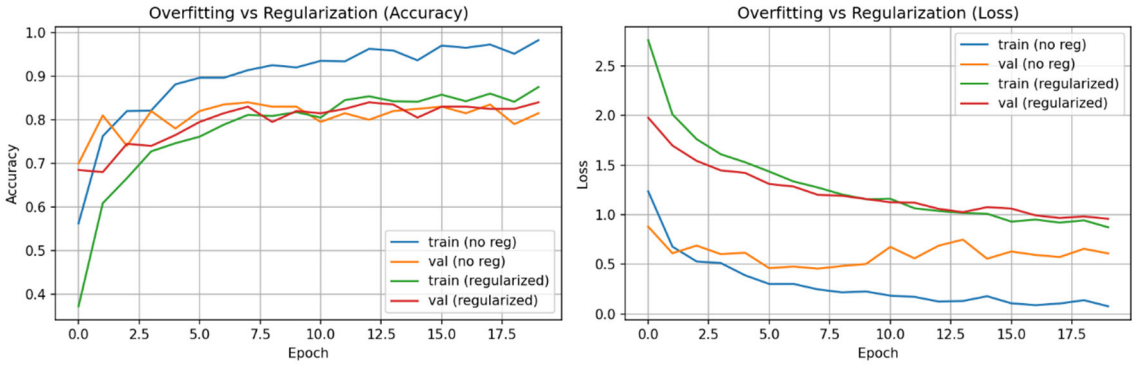
步骤 **7**：过拟合与正则化

使用少量 **Fashion MNIST** 数据**(1000** 个样本**)**训练两个模型进行对比：

(1) 无正则化模型：**Flatten → Dense(512, ReLU) → Dense(512, ReLU) → Dense(10)**，训练 **20** 个 **epoch**；

(2) 正则化模型：**Flatten → Dense(512, ReLU, L2** 正则化**) → Dropout(0.5) → Dense(512, ReLU, L2** 正则化**) → Dropout(0.5) → Dense(10)**。

通过对比两个模型的训练集和验证集准确率/损失曲线，观察过拟合现象及正则化的抑制效果。

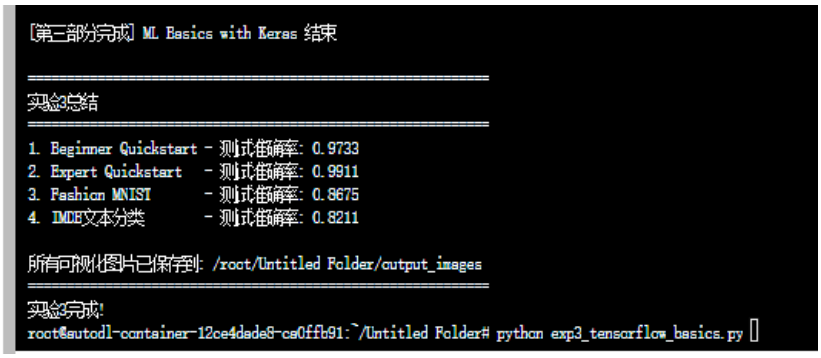


[截图 10：过拟合与正则化对比图]

3.4 实验结果

任务　测试准确率/指标　　备注

MNIST 手写数字分类 (Beginner)	97.33%	Sequential API
MNIST 手写数字分类 (Expert/CNN)	99.11%	Functional API + 自定义循环
Fashion MNIST 图像分类	86.75%	基本图像分类
IMDB 影评情感分析	82.11%	文本分类



[截图 11：实验 3 最终总结]

实验总结：

通过本次实验，我系统地学习了 **TensorFlow** 和 **Keras** 的基础使用方法，掌握了以下关键技能：

1．模型构建：学会了使用 **Sequential API** 和 **Functional API** 构建神经网络，理解了两种 **API** 的适用场景。**Sequential API** 适合简单的线性堆叠模型，而 **Functional API** 更适合复杂的多输入多输出架构。

2．自定义训练：通过 **Experts** 部分的学习，掌握了使用 **tf.GradientTape** 和自定义训练循环的方法，这为实现更复杂的训练逻辑（如 **GAN**、强化学习等）奠定了基础。

3．多任务学习：完成了图像分类、文本分类和回归预测三种基本机器学习任务，理解了不同任务的特点和模型设计要点。

4．正则化技术：通过对比实验直观理解了过拟合现象，掌握了 **L2** 正则化和 **Dropout** 两种常用的正则化方法。

5. **GPU** 加速：成功配置了 **GPU** 环境，体会到 **GPU** 在深度学习训练中的显著加速效果。

参考文献：

[1] TensorFlow. Quickstart for beginners. https://www.tensorflow.org/tutorials/quickstart/beginner

[2] TensorFlow. Quickstart for experts. https://www.tensorflow.org/tutorials/quickstart/advanced

[3] TensorFlow. ML basics with Keras. https://www.tensorflow.org/tutorials/keras

[4] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.

[5] Xiao H, Rasul K, Vollgraf R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms[J]. arXiv preprint arXiv:1708.07747, 2017.

[6] Maas A L, Daly R E, Pham P T, et al. Learning word vectors for sentiment analysis[C]//Proceedings of the 49th annual meeting of the association for computational linguistics. 2011: 142-150.